Student Dropout Prediction - Decision Tree

Kelvin Mai

Spring 2026

1. Introduction

The goal of this analysis is to predict whether a student will drop out (Dropout = 1) or be retained (Dropout = 0) using a Decision Tree classifier. The dataset contains 10,000 student records with 18 predictor variables covering demographics, academic performance, and behavioral factors. We evaluate model stability by testing different tree depths and performing 5-fold cross-validation across multiple random seeds.

2. Load Libraries

```r
library(ggplot2)
library(dplyr)
library(rpart)
library(rpart.plot)
library(pROC)
library(caret)
```

3. Load Data

```r
data <- read.csv("student_dropout_dataset_v3.csv")
dim(data)
```

```
## [1] 10000    19
```

```r
head(data)
```

```
##   Student_ID  Age Gender Family_Income Internet_Access Study_Hours_per_Day
## 1          1 22.1   Male         25000             Yes                3.36
## 2          2 20.7   Male         25000             Yes                4.30
## 3          3 22.4   Male         40183             Yes                4.40
## 4          4 24.4   Male            NA             Yes                  NA
## 5          5 20.5 Female         25319             Yes                4.19
## 6          6 20.5   Male         25000             Yes                4.11
##   Attendance_Rate Assignment_Delay_Days Travel_Time_Minutes Part_Time_Job
## 1            86.1                     2                20.4           Yes
## 2            68.0                     2                44.0            No
## 3            70.9                     0                48.9           Yes
## 4            82.2                     2                38.6            No
## 5            75.7                     1                23.0            No
## 6            89.1                     2                47.1            No
##   Scholarship Stress_Index  GPA Semester_GPA CGPA Semester  Department
## 1          No          5.5 0.96         0.90 0.90   Year 1        Arts
## 2          No          6.8 1.28         1.20 1.19   Year 3 Engineering
## 3          No          5.5 1.68         1.32 1.32   Year 1        Arts
```

```
## 4          No           NA 1.78         1.77 1.77   Year 1          CS
## 5          No          7.0 1.48         0.91 0.87   Year 4    Business
## 6         Yes          6.0 2.52         2.72 2.69   Year 3    Business
##   Parental_Education Dropout
## 1        High School       0
## 2           Bachelor       1
## 3             Master       0
## 4        High School       1
## 5           Bachelor       0
## 6               None       0
```

4. Check Missing Values

```r
missing_counts <- colSums(is.na(data))
missing_counts[missing_counts > 0]
```

```
##       Family_Income Study_Hours_per_Day        Stress_Index
##                 500                 500                 500
```

```r
cat("Blank entries in Parental_Education:",
    sum(data$Parental_Education == "" | is.na(data$Parental_Education)), "\n")
```

```
## Blank entries in Parental_Education: 0
```

Three variables have 500 missing values each: Family_Income, Study_Hours_per_Day, and Stress_Index. Since only 5% of observations are affected per column, we impute with the median. Missing values are assumed to be approximately random (MAR); median imputation is appropriate given the low missingness rate and the non-parametric nature of the decision tree. We also check for blank entries in Parental_Education and replace them with the most frequent category.

5. Data Preprocessing

```r
data$Family_Income[is.na(data$Family_Income)] <-
  median(data$Family_Income, na.rm = TRUE)
data$Study_Hours_per_Day[is.na(data$Study_Hours_per_Day)] <-
  median(data$Study_Hours_per_Day, na.rm = TRUE)
data$Stress_Index[is.na(data$Stress_Index)] <-
  median(data$Stress_Index, na.rm = TRUE)
data$Parental_Education[data$Parental_Education == "" |
                          is.na(data$Parental_Education)] <-
  names(sort(table(
    data$Parental_Education[data$Parental_Education != ""]
  ), decreasing = TRUE))[1]

cat("Remaining NAs:", sum(is.na(data)), "\n")
```

```
## Remaining NAs: 0
```

```r
data$Student_ID      <- NULL

data$Gender             <- as.factor(data$Gender)
data$Internet_Access    <- as.factor(data$Internet_Access)
data$Part_Time_Job      <- as.factor(data$Part_Time_Job)
data$Scholarship        <- as.factor(data$Scholarship)
data$Semester           <- as.factor(data$Semester)
```

```r
data$Department         <- as.factor(data$Department)
data$Parental_Education <- as.factor(data$Parental_Education)
data$Dropout            <- as.factor(data$Dropout)

cat("Dropout distribution:\n")
```

```
## Dropout distribution:
```

```r
table(data$Dropout)
```

```
##
##    0    1
## 7646 2354
```

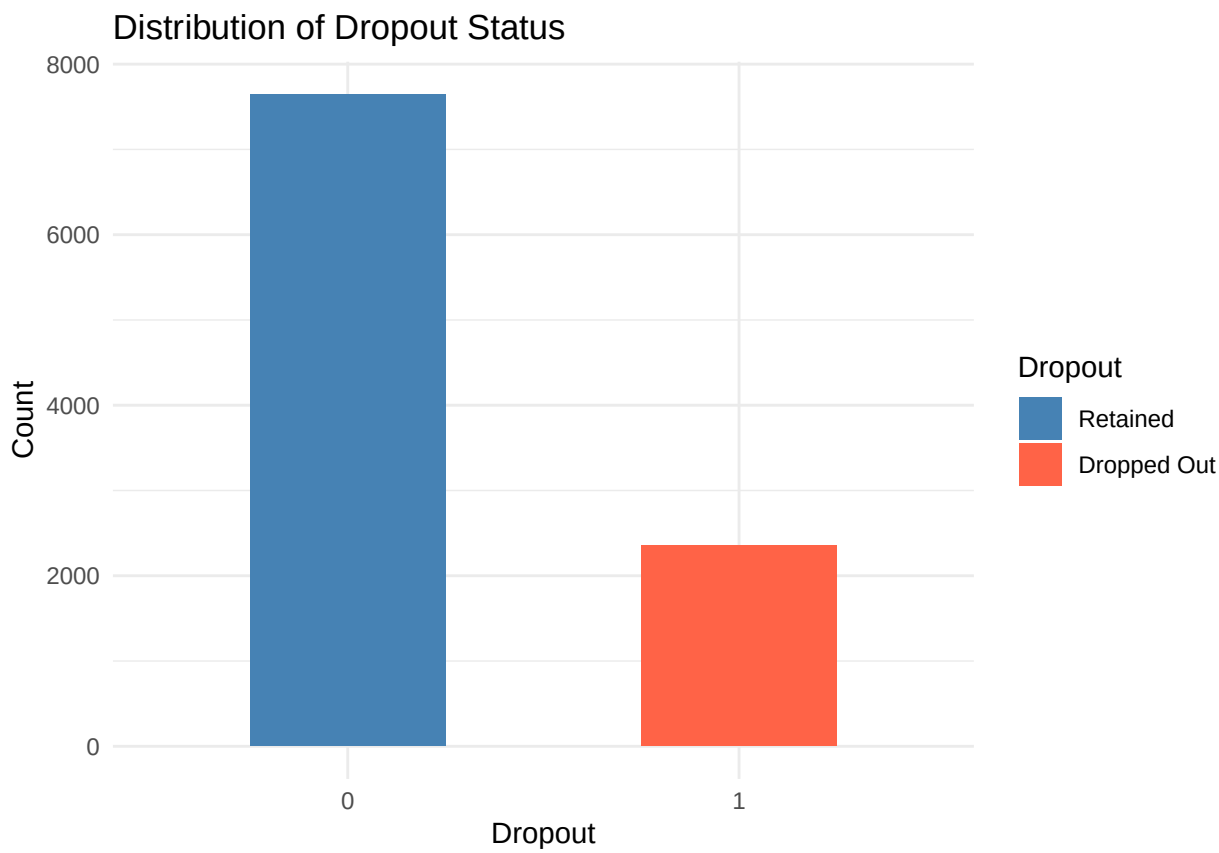
```r
cat("\nDropout rate:", round(mean(data$Dropout == "1") * 100, 2), "%\n")
```

```
##
## Dropout rate: 23.54 %
```

6. Exploratory Data Analysis

6.1 Dropout Distribution

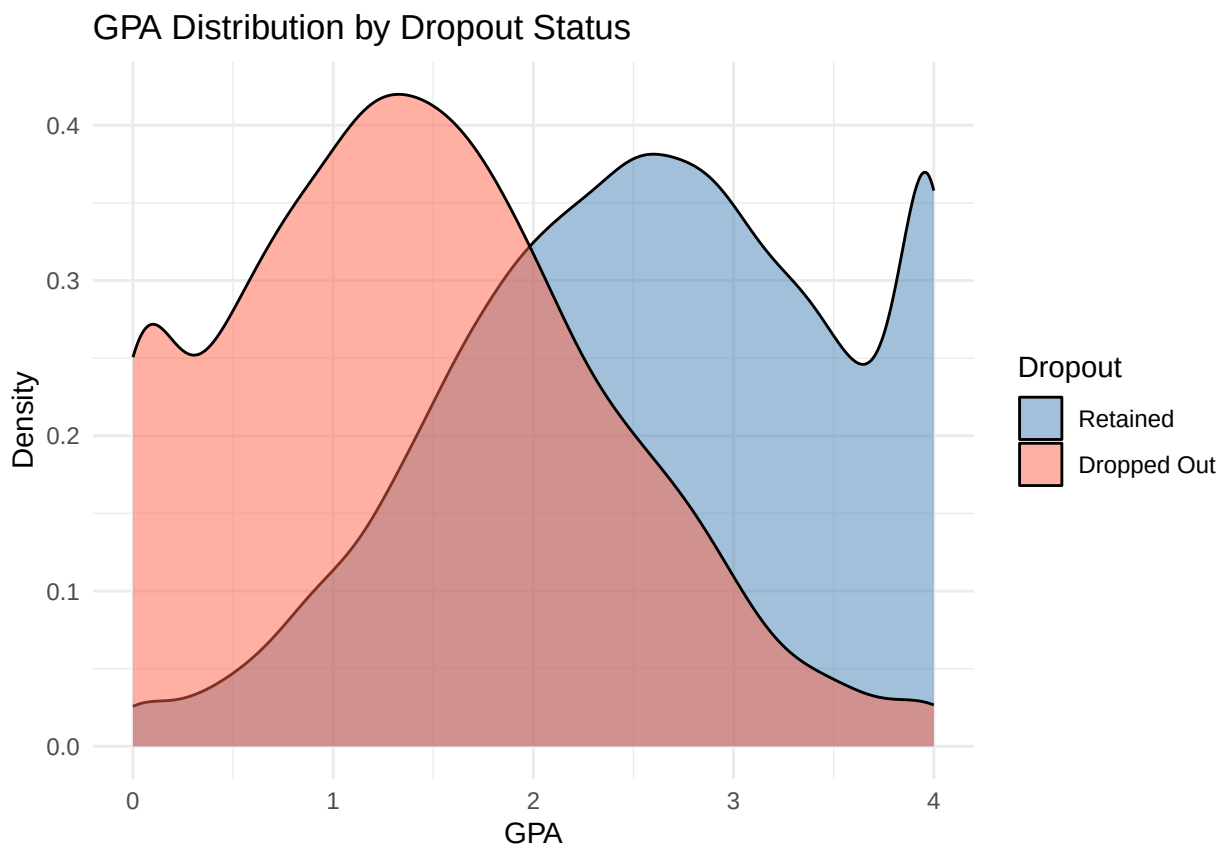
```r
ggplot(data, aes(x = Dropout, fill = Dropout)) +
  geom_bar(width = 0.5) +
  scale_fill_manual(values = c("0" = "steelblue", "1" = "tomato"),
                    labels = c("Retained", "Dropped Out")) +
  labs(title = "Distribution of Dropout Status",
       x = "Dropout", y = "Count") +
  theme_minimal()
```



The dataset is imbalanced: approximately 76.5% of students were retained and 23.5% dropped out. This imbalance is important to keep in mind when interpreting sensitivity, as a model that always predicts "retained" would achieve ~76.5% accuracy without learning anything meaningful.

6.2 GPA Distribution by Dropout Status

```
ggplot(data, aes(x = GPA, fill = Dropout)) +
  geom_density(alpha = 0.5) +
  scale_fill_manual(values = c("0" = "steelblue", "1" = "tomato"),
                    labels = c("Retained", "Dropped Out")) +
  labs(title = "GPA Distribution by Dropout Status",
       x = "GPA", y = "Density") +
  theme_minimal()
```

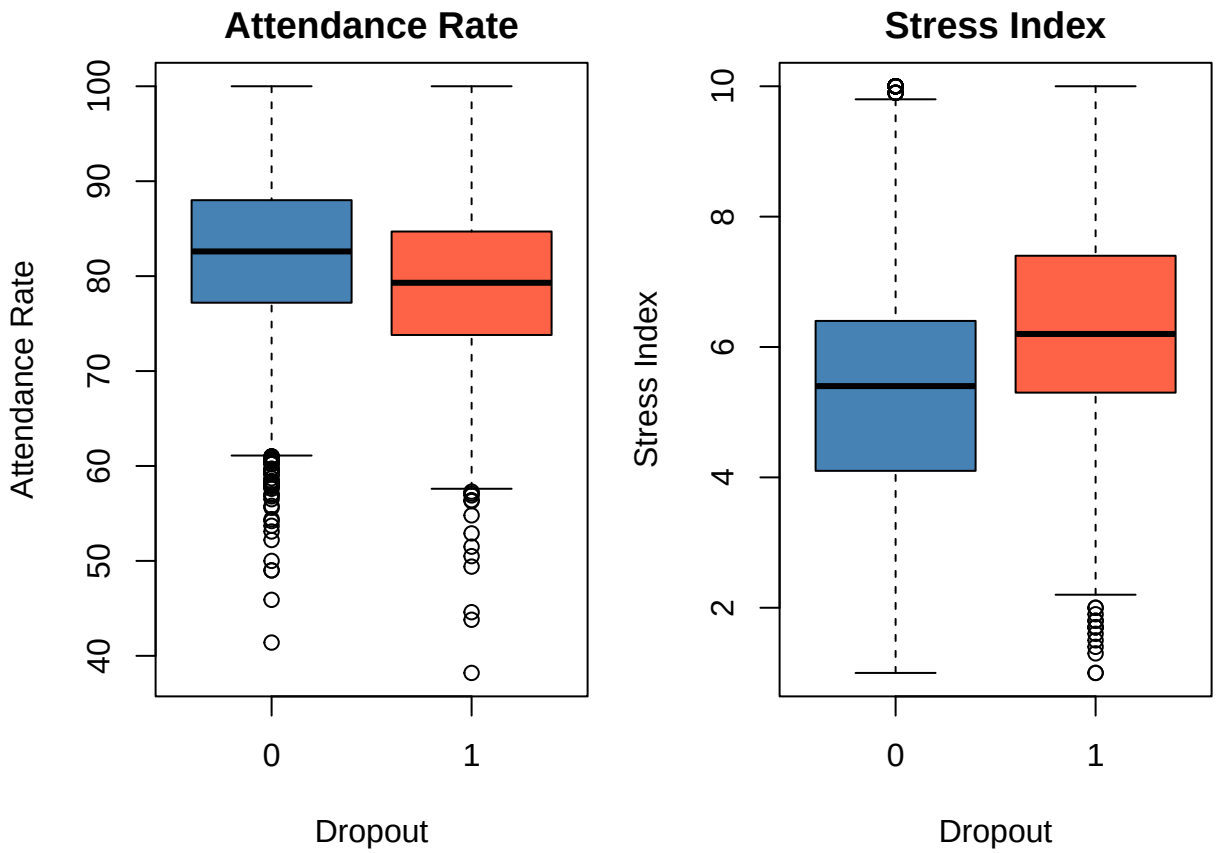


Dropped-out students are concentrated at lower GPA values (roughly below 2.0), while retained students span a wider and higher range. This strong visual separation suggests GPA will be an important predictor in the tree.

Note that GPA, Semester_GPA, and CGPA are all measures of academic performance and are highly correlated. Decision trees handle correlated predictors by selecting whichever split is most informative at each node, so one of the three will dominate and the others will add marginal importance.

6.3 Attendance Rate and Stress Index by Dropout Status

```
par(mfrow = c(1, 2), mar = c(4, 4, 2, 1))
boxplot(Attendance_Rate ~ Dropout, data = data,
        col = c("steelblue", "tomato"),
        main = "Attendance Rate", xlab = "Dropout", ylab = "Attendance Rate")
boxplot(Stress_Index ~ Dropout, data = data,
        col = c("steelblue", "tomato"),
        main = "Stress Index", xlab = "Dropout", ylab = "Stress Index")
```



Attendance rate distributions overlap substantially between retained and dropped-out students, suggesting it is a weaker standalone predictor. Stress index shows a modest shift upward for dropout students, indicating higher stress is associated with dropout risk.

7. Train/Test Split

We randomly split the data into 70% training and 30% testing, following the validation set approach covered in class.

```
set.seed(42)
train      <- sample(1:nrow(data), 0.7 * nrow(data))
train_data <- data[train, ]
test_data  <- data[-train, ]

cat("Training set:", nrow(train_data), "observations\n")
```

```
## Training set: 7000 observations
```

```
cat("Test set:    ", nrow(test_data),  "observations\n")
```

```
## Test set:     3000 observations
```

8. Selecting Tree Depth

Before fitting the final model, we test different values of maxdepth to find the best performing depth. We use `cp = 0.001` to allow the tree to grow at each depth and then read the internal cross-validation error (`xerror`) from the `cptable`.

```r
set.seed(42)
depths        <- c(2, 3, 4, 5, 10, 15)
depth_results <- data.frame(Depth = integer(), CV_Error = numeric())

for (d in depths) {
  model_d  <- rpart(Dropout ~ ., data = train_data, method = "class",
                    control = rpart.control(maxdepth = d, cp = 0.001))
  cp_table <- model_d$cptable
  best_cp  <- cp_table[which.min(cp_table[, "xerror"]), ]
  depth_results <- rbind(depth_results, data.frame(
    Depth    = d,
    CV_Error = round(best_cp["xerror"], 4)
  ))
}

rownames(depth_results) <- NULL
depth_results
```

```
##   Depth CV_Error
## 1     2   0.8567
## 2     3   0.8447
## 3     4   0.8321
## 4     5   0.8435
## 5    10   0.8267
## 6    15   0.8273
```

Depth 4 produces the lowest CV error among the shallow depths. Depths 10 and 15 achieve slightly lower xerror values, but trees of that size are difficult to interpret and risk overfitting to training-specific noise. Depth 5 is marginally worse than depth 4 (difference of 0.0114) but introduces a split on Stress_Index, a behaviorally meaningful variable that GPA alone does not capture. We select **maxdepth = 5** as it offers a reasonable balance between predictive performance, interpretability, and capturing a broader set of risk factors without growing an unwieldy tree.

9. Fit Decision Tree Model

We fit the final model using `cp = 0.001` to be consistent with the depth tuning step. The default `cp = 0.01` prunes too aggressively and would reduce the tree to fewer than 3 splits regardless of depth.

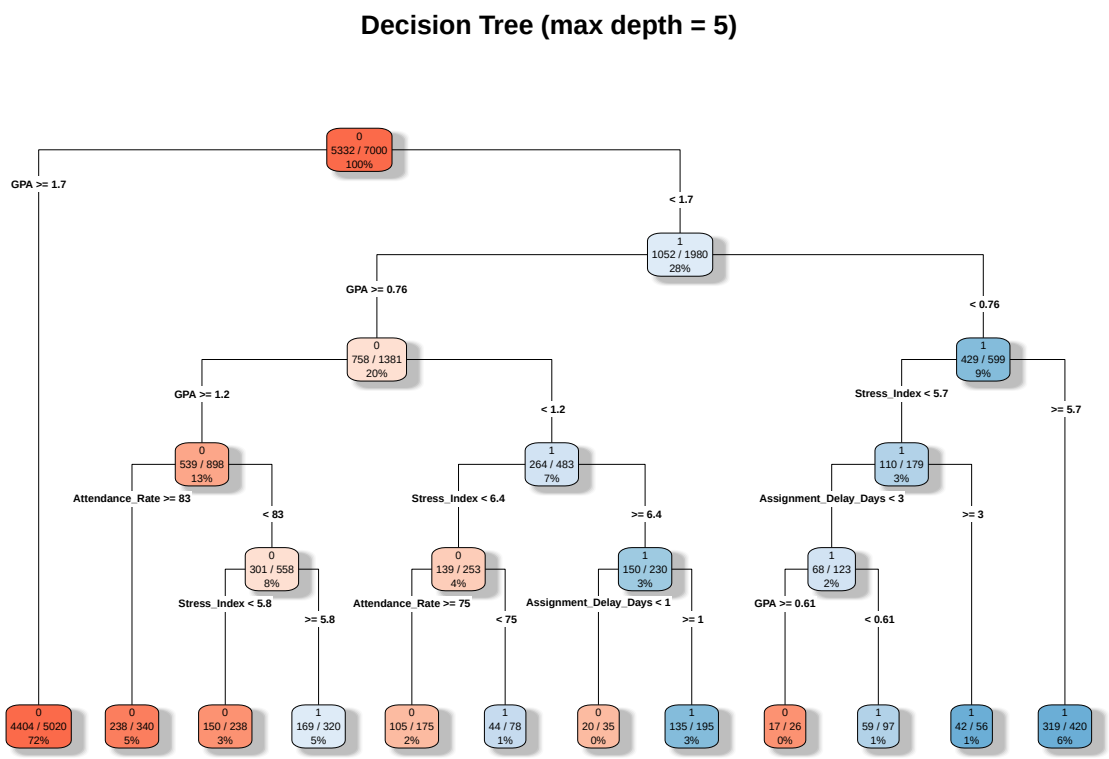
```r
tree_model <- rpart(Dropout ~ ., data = train_data, method = "class",
                    control = rpart.control(maxdepth = 5, cp = 0.001))

printcp(tree_model)
```

```
##
## Classification tree:
## rpart(formula = Dropout ~ ., data = train_data, method = "class",
##     control = rpart.control(maxdepth = 5, cp = 0.001))
##
## Variables actually used in tree construction:
## [1] Assignment_Delay_Days Attendance_Rate       GPA
## [4] Stress_Index
##
## Root node error: 1668/7000 = 0.23829
##
## n= 7000
```

```
##
##          CP nsplit rel error  xerror     xstd
## 1 0.0776379      0   1.00000 1.00000 0.021370
## 2 0.0269784      2   0.84472 0.85851 0.020234
## 3 0.0149880      3   0.81775 0.85252 0.020181
## 4 0.0059952      4   0.80276 0.83573 0.020031
## 5 0.0053957      5   0.79676 0.82374 0.019923
## 6 0.0029976      7   0.78597 0.82614 0.019945
## 7 0.0015987      8   0.78297 0.82794 0.019961
## 8 0.0010000     11   0.77818 0.83034 0.019983
```

10. Visualize the Decision Tree

```r
rpart.plot(tree_model, type = 4, extra = 102,
           box.palette = "RdBu", shadow.col = "gray",
           main = "Decision Tree (max depth = 5)")
```



11. Model Evaluation

```r
pred_class <- predict(tree_model, newdata = test_data, type = "class")
pred_prob  <- predict(tree_model, newdata = test_data, type = "prob")[, 2]

conf_matrix <- table(Predicted = pred_class, Actual = test_data$Dropout)
conf_matrix
```

```
##          Actual
```

```
## Predicted    0    1
##         0 2101  416
##         1  213  270
```

```r
TP <- conf_matrix["1", "1"]
TN <- conf_matrix["0", "0"]
FP <- conf_matrix["1", "0"]
FN <- conf_matrix["0", "1"]

accuracy    <- (TP + TN) / sum(conf_matrix)
sensitivity <- TP / (TP + FN)
specificity <- TN / (TN + FP)
error_rate  <- 1 - accuracy

cat("Accuracy:   ", round(accuracy,    4), "\n")
```

```
## Accuracy:    0.7903
```

```r
cat("Error Rate: ", round(error_rate,  4), "\n")
```

```
## Error Rate:  0.2097
```

```r
cat("Sensitivity:", round(sensitivity, 4), "\n")
```

```
## Sensitivity: 0.3936
```

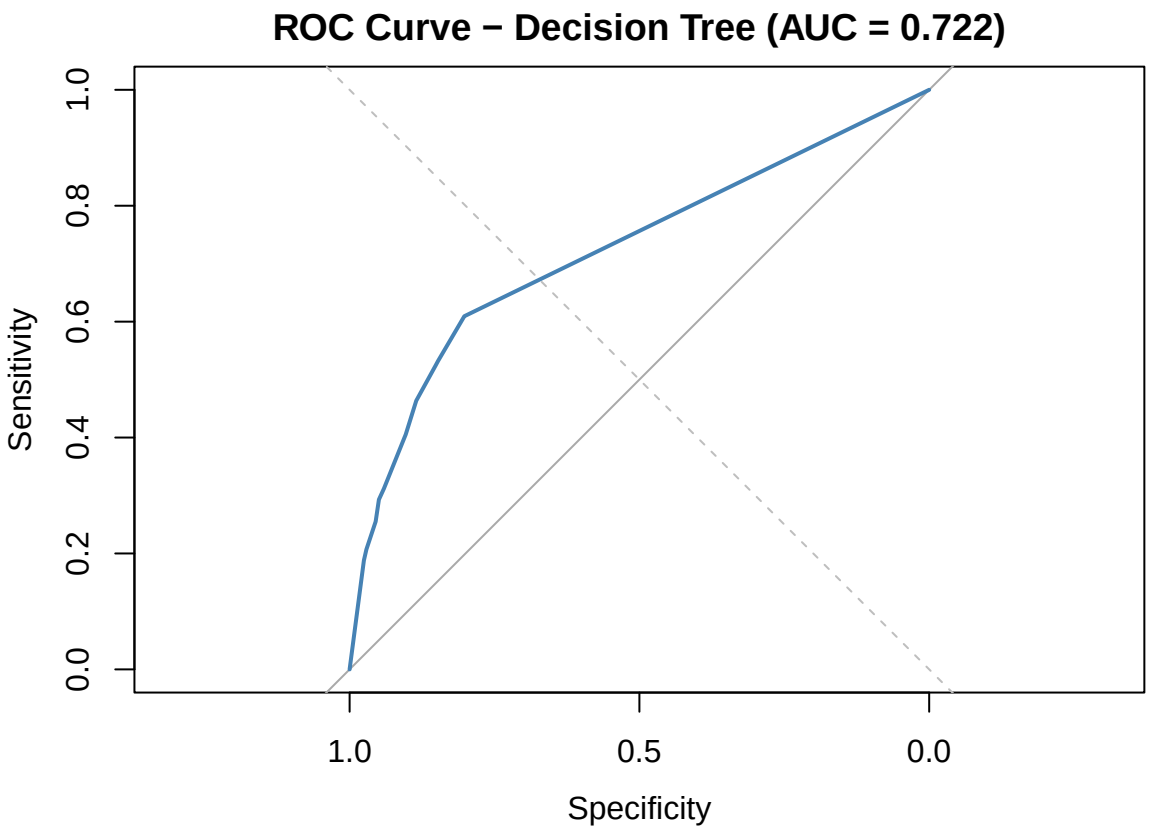
```r
cat("Specificity:", round(specificity, 4), "\n")
```

```
## Specificity: 0.908
```

```r
roc_obj <- roc(as.numeric(test_data$Dropout) - 1, pred_prob)
cat("ROC-AUC:    ", round(auc(roc_obj), 4), "\n")
```

```
## ROC-AUC:     0.7224
```

```r
plot(roc_obj,
     col  = "steelblue", lwd = 2,
     main = paste0("ROC Curve - Decision Tree (AUC = ",
                   round(auc(roc_obj), 3), ")"))
abline(a = 0, b = 1, lty = 2, col = "gray")
```



Accuracy and specificity are reasonable, but sensitivity is low. The model correctly identifies roughly 39% of actual dropout cases, meaning it misses the majority. This is a direct consequence of class imbalance: with roughly 3 retained students for every 1 dropout, the tree is incentivized to predict "retained" more often. The ROC-AUC provides a threshold-independent view of discrimination ability.

12. Resampling with Different Seeds

The validation set approach depends on the random split. We evaluate the model using 5 different random seeds to check stability.

```
seeds <- c(42, 123, 456, 789, 2026)
resample_results <- data.frame(
  Seed        = integer(),
  Accuracy    = numeric(),
  Error_Rate  = numeric(),
  Sensitivity = numeric(),
  Specificity = numeric()
)

for (s in seeds) {
  set.seed(s)
  idx <- sample(1:nrow(data), 0.7 * nrow(data))
  tr  <- data[idx, ]
  te  <- data[-idx, ]

  model  <- rpart(Dropout ~ ., data = tr, method = "class",
                  control = rpart.control(maxdepth = 5, cp = 0.001))
```

```r
  pred_c <- predict(model, newdata = te, type = "class")
  cm     <- table(Predicted = pred_c, Actual = te$Dropout)

  tp <- cm["1", "1"]
  tn <- cm["0", "0"]
  fp <- cm["1", "0"]
  fn <- cm["0", "1"]

  resample_results <- rbind(resample_results, data.frame(
    Seed        = s,
    Accuracy    = round((tp + tn) / sum(cm),   4),
    Error_Rate  = round(1 - (tp + tn) / sum(cm), 4),
    Sensitivity = round(tp / (tp + fn),          4),
    Specificity = round(tn / (tn + fp),          4)
  ))
}

rownames(resample_results) <- NULL
resample_results
```

```
##   Seed Accuracy Error_Rate Sensitivity Specificity
## 1   42   0.7903     0.2097      0.3936      0.9080
## 2  123   0.8023     0.1977      0.3835      0.9246
## 3  456   0.7970     0.2030      0.3533      0.9326
## 4  789   0.7883     0.2117      0.3893      0.9078
## 5 2026   0.8017     0.1983      0.3129      0.9504
```

```r
cat("\n=== Average Performance Across 5 Seeds ===\n")
```

```
##
## === Average Performance Across 5 Seeds ===
```

```r
cat("Accuracy:   ", round(mean(resample_results$Accuracy),    4),
    " (SD:", round(sd(resample_results$Accuracy),    4), ")\n")
```

```
## Accuracy:    0.7959  (SD: 0.0064 )
```

```r
cat("Error Rate: ", round(mean(resample_results$Error_Rate),  4),
    " (SD:", round(sd(resample_results$Error_Rate),  4), ")\n")
```

```
## Error Rate:  0.2041  (SD: 0.0064 )
```

```r
cat("Sensitivity:", round(mean(resample_results$Sensitivity), 4),
    " (SD:", round(sd(resample_results$Sensitivity), 4), ")\n")
```

```
## Sensitivity: 0.3665  (SD: 0.0339 )
```

```r
cat("Specificity:", round(mean(resample_results$Specificity), 4),
    " (SD:", round(sd(resample_results$Specificity), 4), ")\n")
```

```
## Specificity: 0.9247  (SD: 0.0179 )
```

Accuracy and specificity are stable across seeds (low SD). Sensitivity shows more variability, which is expected given the class imbalance: small differences in how the minority class is distributed across splits can noticeably affect the true positive rate.

13. Cross-Validation

We use 5-fold cross-validation across multiple seeds to further validate model stability, as introduced in class.

```
cv_seeds   <- c(42, 123, 456, 789, 2026)
cv_results <- data.frame(Seed = integer(), CV_Error = numeric())

for (s in cv_seeds) {
  set.seed(s)
  folds       <- createFolds(data$Dropout, k = 5, list = TRUE)
  fold_errors <- c()

  for (fold in folds) {
    tr     <- data[-fold, ]
    te     <- data[fold, ]
    model  <- rpart(Dropout ~ ., data = tr, method = "class",
                    control = rpart.control(maxdepth = 5, cp = 0.001))
    pred_c <- predict(model, newdata = te, type = "class")
    fold_errors <- c(fold_errors, mean(pred_c != te$Dropout))
  }

  cv_results <- rbind(cv_results, data.frame(
    Seed     = s,
    CV_Error = round(mean(fold_errors), 4)
  ))
}

rownames(cv_results) <- NULL
cv_results
```

```
##   Seed CV_Error
## 1   42   0.1998
## 2  123   0.2048
## 3  456   0.2004
## 4  789   0.2037
## 5 2026   0.1998
```

```
cat("\nMean CV error:", round(mean(cv_results$CV_Error), 4), "\n")
```

```
##
## Mean CV error: 0.2017
```

```
cat("SD of CV error:",
    format(round(sd(cv_results$CV_Error), 4), scientific = FALSE), "\n")
```

```
## SD of CV error: 0.0024
```

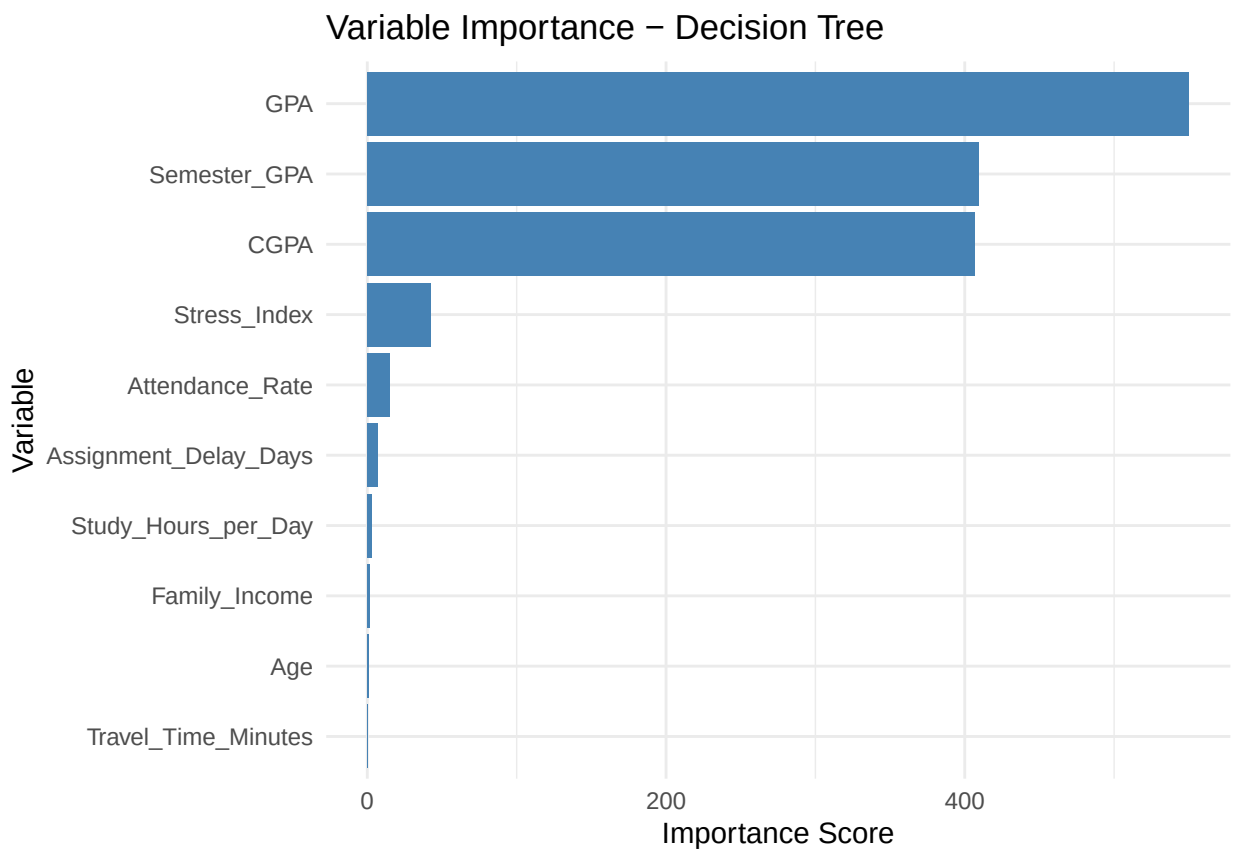
The very low standard deviation across seeds confirms that the 5-fold CV error estimate is stable and not sensitive to how the folds are assigned.

14. Variable Importance

Variable importance in `rpart` reflects how much each variable reduces node impurity across all splits where it was used. Only variables selected in at least one split appear.

```
importance    <- tree_model$variable.importance
importance_df <- data.frame(
  Variable   = names(importance),
```

```r
  Importance = as.numeric(importance)
) %>% arrange(desc(Importance))

ggplot(importance_df,
       aes(x = reorder(Variable, Importance), y = Importance)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(title = "Variable Importance - Decision Tree",
       x = "Variable", y = "Importance Score") +
  theme_minimal()
```



GPA dominates variable importance by a large margin, followed by Semester_GPA and CGPA. These three variables all measure academic performance and are highly correlated with one another. The tree effectively uses them interchangeably depending on the split, so their combined dominance reflects a single underlying signal: academic performance is the strongest predictor of dropout. Variables like Gender, Scholarship, and Department contributed negligibly and were not selected in any split.

Stress_Index appears with modest importance, capturing a behavioral dimension that GPA alone does not. This is the primary reason for selecting depth 5 over depth 4.

15. Conclusion

The Decision Tree model with maxdepth = 5 and cp = 0.001 achieved an accuracy of 0.79 and a ROC-AUC of 0.722 on the test set (seed = 42). Across 5 random seeds, the model was stable with a mean accuracy of 0.796 (SD = 0.0064) and a mean error rate of 0.204 (SD = 0.0064). The 5-fold cross-validation gave a mean CV error of 0.2017 (SD = 0.0024).

GPA was the dominant predictor of dropout, with Semester_GPA and CGPA also contributing. Because all three measure academic performance and are highly correlated, their combined importance reflects a single underlying factor rather than three independent signals. Stress_Index contributed modestly at deeper splits, capturing a behavioral dimension beyond grades alone.

Sensitivity averaged approximately 37% across seeds, meaning the model missed roughly two-thirds of actual dropout cases. This is a known limitation of decision trees trained on imbalanced data: because the majority class (retained) is approximately three times larger, the tree minimizes overall error by predicting retained more often. Addressing this in a future iteration would require either adjusting the classification threshold, applying class weights, or using an ensemble method such as a random forest that is more robust to class imbalance.